NH-3779 — Metro Underground Location (POC)

Requirements, what is achieved, technologies used, improvements, options to explore and limitations

25 September 2026

1. About this POC

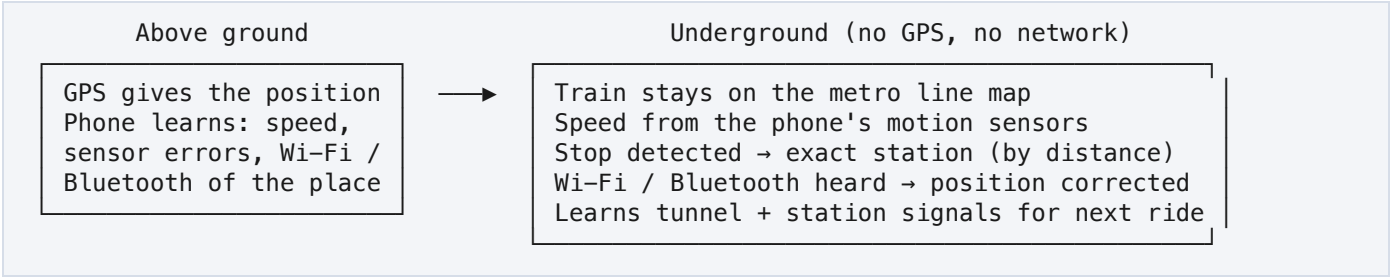
Item	Value
JIRA ticket	**NH-3779**
Goal	Know the position (latitude/longitude, line, station) of a train **inside metro tunnels and stations**, where GPS and mobile network do not work
Device	A **high-end Android phone mounted on the train**
Approach	Everything runs **on the phone**: offline, no server
Status	POC built. **Validation on a real metro is the next step.**

The idea in one sentence: the phone knows the metro line map, measures the train's speed and stops with its own sensors, recognises each station by the distance travelled, and learns the Wi-Fi and Bluetooth signals of stations and tunnels so that every ride helps the next one.

2. Requirements

#	Requirement	In simple words
R1	Location inside **underground metro tunnels and stations**	Must work in both places
R2	**No access to the train's own location system**	The phone works alone
R3	Estimate from **coverage detail (can be trained)**	Use the radio signals found there, learnt over time
R4	Use **additional beacons**	Use Bluetooth beacons in stations / tunnels
R5	Use **all Android built-in features**	Every sensor and radio of the phone
R6	Check **commercial solutions**	See whether a product can be applied

3. How it works



1. **Before going underground** the phone saves the metro lines of the current city and uses GPS.
2. **In the tunnel** the train's position moves along the line using the speed measured by the phone's motion sensors.
3. **At every stop** the phone checks the train really stopped, works out which station it is from the distance travelled, and places the train exactly at that station.
4. **Wi-Fi and Bluetooth** signals heard underground are compared with what the phone has learnt before, and correct the position when they clearly know better.
5. **After every ride** the phone has learnt more (signals of stations and tunnels, speed of every run), so the next ride is better.

4. What has been achieved

4.1 Per requirement

#	Achieved	Status
R1	Position in tunnels and stations (details in 4.2)	✔ Built
R2	Nothing from the train or the operator is needed; metro line maps come from OpenStreetMap (free)	✔ Built
R3	Wi-Fi and Bluetooth "fingerprints" of stations and tunnels are learnt automatically on each ride; Android/Google network location is also used when a network is available	✔ Built (mobile-cell coverage: can be added, see 7)
R4	Bluetooth beacons (iBeacon, AltBeacon, Eddystone) are read automatically and used as checkpoints	✔ Built (works where beacons exist)
R5	GPS, network location, accelerometer, gyroscope, orientation sensor, compass, barometer, Wi-Fi scan, Wi-Fi RTT, Bluetooth LE	✔ Built (more can be added, see 7)
R6	Commercial products checked; none fits a phone on a train in metro tunnels (see 8.2)	✔ Evaluated

4.2 In tunnels and in stations

Place	What the POC does
Tunnel — train moving	Keeps the train on the correct line; measures speed without GPS; moves the position along the line; knows the direction of travel; corrects with learnt Wi-Fi/Bluetooth

Place	What the POC does
Tunnel — train stops between stations	Recognises a stop in the tunnel (e.g. at a signal) and does not mistake it for a station
Station — train stops	Places the train exactly at the platform and identifies the right station
Station — train passes without stopping	Recognises the station was passed (express trains)
Going underground	Uses the last GPS position at the tunnel entrance and continues along the line
Starting underground	Finds the first position from learnt Wi-Fi/Bluetooth of the station
End of the line	Detects the train reversing and changes direction
Any city	Downloads the current city's metro lines automatically when online (anywhere in the world); 18 Indian cities are also included in the app for first use without network

5. Technologies used

Technology	What it does in the POC	When it is used
Metro line map (OpenStreetMap)	Shape of each line and station positions; keeps the train on the track	Always
GPS / GNSS	True position above ground; teaches speed and sensor errors	Above ground, tunnel entrances
Android network location (Google)	Position from Wi-Fi / mobile networks	Underground, only if a network is available
Accelerometer	Train speed: acceleration added up since the last stop, where speed is exactly zero	Whole ride
Gyroscope + orientation sensor	Turns sensor readings into real directions; measures the train's turns in curves	Whole ride
Compass (magnetometer)	Direction of travel (its error from train motors is learnt and corrected)	Departures
Barometer	Detects going from above ground to underground	Station entrances / ramps
Wi-Fi scanning	Signal "fingerprints" of stations and tunnels	Every 30 s (Android limit)
Wi-Fi RTT	Distance to Wi-Fi access points that support it	Where access points support it
Bluetooth LE	Beacons as checkpoints; part of fingerprints	Continuously
Station-stop detection + distance "odometer"	Exact station at every stop; tells stations from signal stops	Every stop
Hidden Markov Model (probability method)	Combines all evidence and keeps every possible position on the track	When Wi-Fi / Bluetooth is heard
Ride memory	Remembers the speed of every station-to-station run (see 6)	From the second ride
Offline storage	Line maps, learnt signals and run speeds kept on the phone	Always

6. How the phone learns

What is learnt	From	Kept
Wi-Fi / Bluetooth fingerprints of **stations**	Signals heard while the position is certain (good GPS, or standing at a confirmed station)	Saved on the phone (`radio_map.txt`)
Wi-Fi / Bluetooth fingerprints **inside tunnels**	Signals heard between two confirmed stations, placed along the tunnel by distance travelled	Saved on the phone (`radio_map.txt`)
Speed of each run (ride memory)	Moving time between two neighbouring confirmed stations	Saved on the phone (`ride_memory.txt`)
Typical speed of the line	GPS speed and station-to-station times	During the ride
Accelerometer error	Readings while the train stands at a station	During the ride
Compass error	Direction of acceleration vs. known direction of travel	During the ride

Ride memory in short: for each run (line, from-station, to-station) the phone remembers the track length and the average moving time over the last rides. From these it works out the run's cruising speed (assuming normal acceleration and braking). The next time the train leaves that station, this speed is used as the expected speed, and the phone relies more on its own speed estimate on that run.

Learning only from certain positions: the phone learns only at confirmed stops or with good GPS — never from its own guesses — and skips anything unrealistic (e.g. a long hold in a tunnel).

7. What can be improved (on the phone)

#	Improvement	Benefit
1	**Mobile-cell coverage fingerprints** — cell IDs and signal strengths are available from Android without internet	A "coverage" signal wherever the metro has mobile coverage (R3)
2	**Android Activity Recognition** ("in vehicle" / "still")	Extra check for stops and departures
3	**Save the ride-level learning** (line speed, sensor and compass errors) between app restarts	Better from the first minute of every ride
4	**Exclude learnt data from Android cloud backup**	Keeps all learnt data strictly on the phone
5	**Tuning with real ride data** for the mounted position	Accuracy matched to the real train and phone
6	**Simple offline map screen** (line, moving dot, ± error)	Easier to see and explain the result

8. What can be explored

8.1 Further technologies

Option	Idea	Needs first
Offline AI module (TensorFlow Lite, free)	Learn the train's speed from its vibration; recognise stopped / accelerating / braking; recognise stations from their signals. Trained on a PC from our own ride recordings (GPS labels the data automatically above ground) and run on the phone's AI chip, offline	Recording raw sensor data on real rides
Magnetic-field fingerprints	Steel structures and track give a local magnetic "signature"; a mounted phone keeps a fixed orientation	Check it repeats on the same line
Other beacon formats (e.g. Waze beacons)	More checkpoints where such beacons exist	Beacons present on the line
Wi-Fi RTT in stations	Metre-level distance to access points	Station access points that support it

8.2 Commercial solutions (R6)

Solution	Result
IndoorAtlas, Pointr, Situm, Mapsted, HERE Indoor	Not free for production; need their cloud and a survey of every venue; built for people walking in buildings, not for trains in tunnels → **not applicable**
Google Maps underground positioning, transit apps (Citymapper, Moovit)	Work in metros, but have no SDK / API for other apps → **not applicable**
Google Fused Location	**Already used** when a network is available
Tunnel / commercial Bluetooth beacons	**Already supported** where they exist

Conclusion: no commercial product can be plugged in; the in-house POC is the right base.

9. Limitations

- **No GPS underground** — underground positions are estimates, shown with their source and ± error.
- **First ride on a line** is the learning ride; radio and run-speed learning help from the second ride.
- **Learning is per phone** (no server): each mounted phone learns its own line.
- **Express trains on a never-ridden line:** while passing a skipped station the shown position can wait at that station until the next stop.
- **Wi-Fi scans are limited by Android** (4 per 2 minutes); Bluetooth fills the gaps.
- **Radio help depends on the line:** Wi-Fi / Bluetooth must exist in stations or tunnels.
- **Metro line map needed:** the phone must have been online once in the city (or the city is included in the app).
- **Line-level learning (speed, sensor errors) restarts each session** until improvement 7.3.
- **Learnt data can currently go into Android cloud backup** until improvement 7.4.

10. Status and next step

Stage	Status
POC design and implementation	✔ Done
Documentation (HLD, Technical Design, Technology Guide, this POC summary)	✔ Done
Validation on a real metro with the phone mounted on the train	□ Next step
Improvements (section 7) and explorations (section 8)	□ After validation